



VO Computernetzwerke – Kapitel 6

SS 2026

Armin Veichtlbauer

6. Transmission Control Protocol

Überblick

TCP Grundlagen:

Wie funktioniert TCP? Welche Headerfelder hat TCP und wozu dienen sie? Wie funktioniert der TCP 3-way Handshake?

ARQ:

Wie funktioniert ARQ? Welche ARQ Verfahren können unterschieden werden? Was sind Schiebefensterverfahren und wie funktionieren sie?

Überlaststeuerungsmechanismen in TCP:

Welches ARQ Verfahren verwendet TCP? Wie kann das für Überlaststeuerung verwendet werden? Wie funktionieren die Berechnungen der Fenstergrößen?

6.1 TCP Grundlagen

Eigenschaften

- Das Transmission Control Protocol ist definiert in RFC 793 (ergänzt durch RFCs 1122 und 1323) und stellt eine End-to-end Verbindung über ein IP Netz her
- TCP wird verwendet als Transportdienst im Internet für alle verteilten Anwendungen, die einen zuverlässigen verbindungsorientierten Transportdienst benötigen (z.B. File Transfer, File Sharing, Email, Web browsing, etc.)
- Das Protokoll implementiert Flow Control und Congestion Control Algorithmen
- Es transportiert einen Bytestrom über einen Virtual Circuit; dieser Bytestrom wird beim Sender in Segmente unterteilt und beim Empfänger wieder reassembliert

6.1 TCP Grundlagen

Zuverlässigkeit von TCP

- TCP bietet die Zuverlässigkeit (Reliability) durch verschiedene Mechanismen:
 - Bestätigung des Empfangs eines Segments (durch ACK)
 - Erneute Übertragung eines nicht zeitgerecht zugestellten (wird durch ein Timeout ermittelt) Segments (durch ARQ) → **Retransmission**
 - Erkennung von Duplikaten (durch eindeutige Sequenznummern)
 - Erhaltung der Segment-Reihenfolge (durch aufsteigende Sequenznummern)
 - Sicherung der Integrität durch eine Prüfsumme (wie UDP)
- TCP bietet aber keine **Angriffssicherheit** (Security) → Keine Verschlüsselung und auch keine Authentifizierung!

6.1 TCP Grundlagen

TCP Header

0 - 3	4 - 7	8 - 11	12 - 15	16 - 19	20 - 23	24 - 27	28 - 31
Source Port				Destination Port			
Sequence Number (SN)							
Acknowledge Number (AN)							
Offset	Reserved	Flags		Window (WND)			
Checksum				Urgent Pointer			
Options und Padding (optional)*							

* Options können auch mehrere 32 bit Worte umfassen (aber immer nur ganze Worte)

6.1 TCP Grundlagen

Bedeutung der Headerfelder

- Source Port und Destination Port: Wie bei UDP
- Sequence Number (SN): Nummer des ersten Bytes eines TCP Segments im originalen Bytestrom
- Acknowledge Number (AN): Nummer des ersten noch nicht bestätigten Bytes im originalen Bytestrom
- Offset: Länge des TCP Headers (in Worten, also in Vielfachen von 4 Byte → ohne Options also 5)
- Flags: Steuerungsbits für den TCP Protokollablauf (URG, ACK, PSH, RST, SYN, FIN)
- Window (WND): Größe des Empfangsfensters (so viele Bytes dürfen unbestätigt gesendet werden)
- Checksum: Wie bei UDP (auch hier wird ein Pseudoheader verwendet)
- Urgent Pointer: Länge des „Urgent Bereichs“ am Beginn der TCP Nutzdaten (bei gesetztem URG Flag)

6.1 TCP Grundlagen

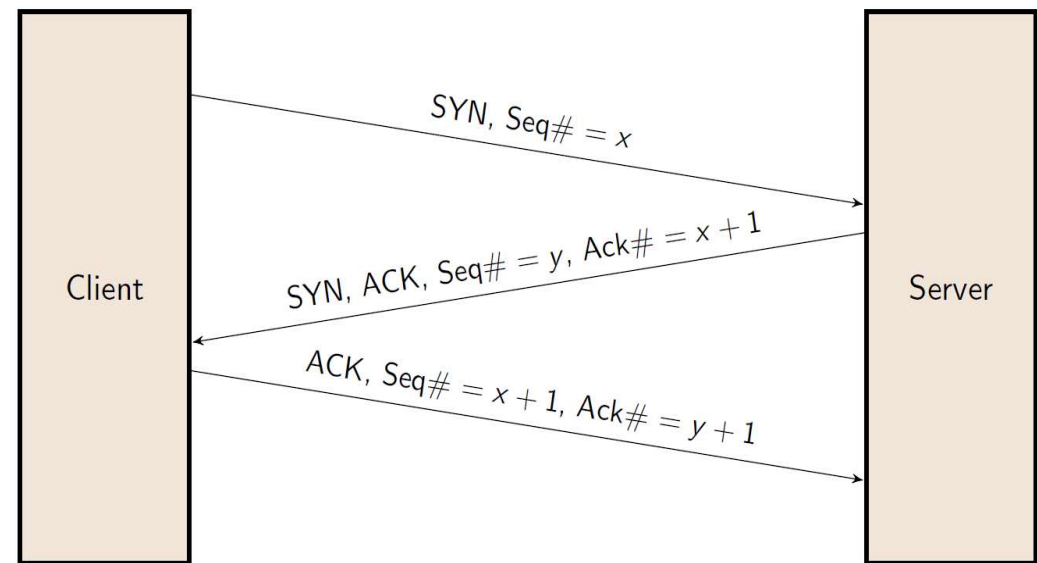
Flags (Steuerungsbits)

- URG (Urgent Flag): Ermöglicht der Anwendung, die Nutzdaten dieses Segments bis zum Urgent Pointer sofort zu lesen und dann entsprechend zu reagieren
- ACK (Acknowledge Flag): Bestätigt die korrekte Ankunft von Daten entsprechend dem Wert in der Acknowledged Nummer
- PSH (Push Flag): Zeigt der Anwendung an, dass Daten da sind, die sie abholen sollte, um die Bearbeitung zu beschleunigen (vor allem für zeitkritische Anwendungen)
- RST (Reset Flag): Setzt eine Verbindung ohne Handshake zurück, etwa bei technischen Problemen; wird auch bei Abweisung unerwünschter Verbindungen gesetzt
- SYN (Synchronize Flag): Zeigt eine Anfrage um Verbindungsaufbau an
- FIN (Finish Flag): Beendet die Verbindung mit üblichen FIN Handshake

6.1 TCP Grundlagen

TCP Verbindungsaufbau

- Anfrage zu Verbindungsaufbau
 - Initiale SN des Clients (x)
- Antwort auf Anfrage
 - Initiale SN des Servers (y), AN des Servers ($x+1$)
- Bestätigung des Aufbaues
 - SN des Servers ($x+1$), AN des Clients ($y+1$), Daten



TCP 3-Way Handshake

6.2 Automated Repeat Request

Grundidee

- Ziele
 - Wenn ein Segment richtig empfangen wird, wird es bestätigt (Acknowledge)
 - Wenn ein Segment nicht richtig empfangen wird, wird es nochmals gesendet
 - Dazu müssen verlorene Segmente durch Nummerierung erkannt werden
- Fehlerentdeckung
 - Implizite Fehlerentdeckung: durch Ziehen eines Time-Outs
 - Explizite Fehlerentdeckung: durch das Zurückschicken eines NAK (etwa wegen CRC Fehlern) → Achtung: NAK funktioniert nicht bei Segmentverlust

6.2 Automated Repeat Request

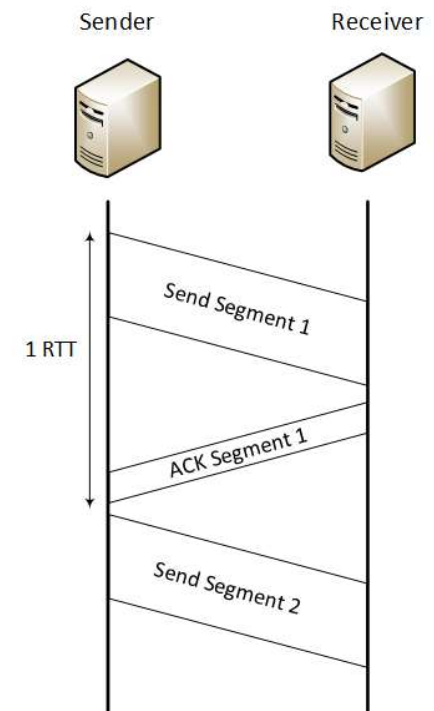
Arten von ARQ Verfahren

- Stop and Wait
 - Erst nach Bestätigung darf ein weiteres Segment gesendet werden
 - Das kann als Spezialfall der Schiebefensterprotokolle gesehen werden
- Schiebefenster Verfahren (Sliding Window)
 - Durch einen Kreditmechanismus wird festgelegt, wie viele Segmente bzw. wie viele Bytes ohne Bestätigung gesendet werden dürfen → Fenstergröße (in Segmenten bzw. in Byte oder in Vielfachen der Maximum Segment Size)
 - Diese Verfahren können weiter unterteilt werden (unterscheidbar nur im Fehlerfall): GoBackN, Selective Repeat

6.2 Automated Repeat Request

Stop and Wait ARQ

- Funktionsweise
 - Erst nach Erhalt des ACKs kann ein weiteres Segment gesendet werden
 - Nach Ziehen eines Timeouts wird das Segment als verloren interpretiert und erneut gesendet
- Durchsatzbewertung
 - Durchsatz ist sehr schlecht, weil sehr hohe Wartezeiten entstehen (z.B. bei RTTs von 100 ms und MSS von 1460 Byte: $10 \text{ Segmente/s} = 14\,600 \text{ Byte/s}$)



Wartezeiten bei Stop&Wait

6.2 Automated Repeat Request

Prinzip der Schiebefensterverfahren

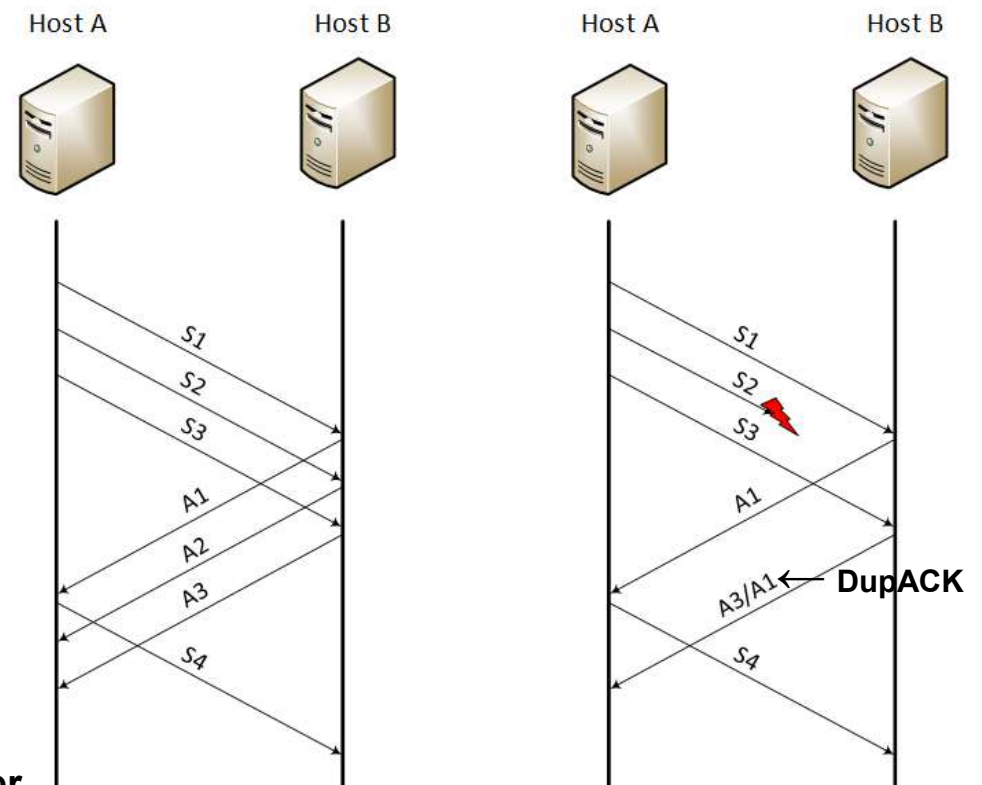
- Mehrere Segmente werden ohne Warten unmittelbar hintereinander gesendet
- Wie viel gesendet werden darf, hängt von der Fenstergröße ab (siehe oben)
- Pro gesendetem Segment werden also Kredite entsprechend der Segmentgröße (nur Nutzdaten!) verbraucht
- Wenn kein Kredit mehr vorhanden ist, wird unterbrochen
- Durch Eintreffen von Bestätigungen (ACKs) werden weitere Kredite verfügbar und das Senden kann fortgesetzt werden

6.2 Automated Repeat Request

Ablauf mit und ohne Fehler

- Die Bestätigungen können kumulativ (alles bis inkl. Segment n) oder selektiv (genau Segment n) erfolgen
- Beim Ablauf ohne Fehler macht es keinen Unterschied → jede **neue** Bestätigung macht einen Kredit frei
- Lücken führen bei kumulativer Bestätigung zu **DupACKs**

Sliding Window Ablauf mit (rechts) und ohne (links) Fehler



6.2 Automated Repeat Request

Fehlerbehandlung in Schiebefensterverfahren

- Bei Ziehen eines Timeouts (eventuell bei Empfang eines NAKs) wird das entsprechende Segment verloren gegeben
 - Das Segment muss dazu identifiziert werden; TCP verwendet dazu die Sequenznummer
 - Dann erfolgen Retransmissions – Diese brauchen **keine** neuen Kredite!
- Ist die Retransmission erfolgreich, kann der Empfänger das Segment verarbeiten (bei TCP also in den Bytestrom einordnen) und der Sender erhält ein ACK
 - Das ACK muss dieses Segment implizit (bei kumulativer Bestätigung) oder explizit (bei selektiver Bestätigung) identifizieren können

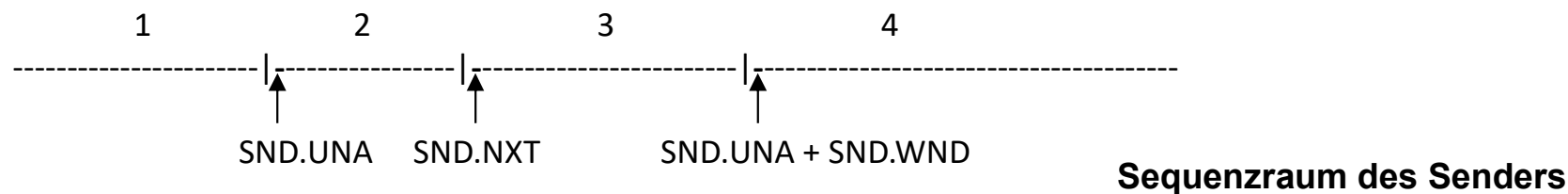
6.2 Automated Repeat Request

Varianten von Schiebefensterverfahren

- GoBackN
 - Falls ein Fehler auftritt, wird ab dem verlorenen Segment alles wiederholt
 - GoBackN wird für größere Fenster zunehmend ineffizient, da viele bereits richtig übertragene Segmente erneut übertragen werden müssen
- Selective Repeat
 - Falls ein Fehler auftritt, wird nur das verlorene Segment wiederholt
 - Wenn der Empfänger einen Puffer für die vorher bereits korrekt empfangenen Segmente besitzt, kann er nach erfolgreicher Retransmission die Lücken füllen und die Reihenfolge wieder herstellen

6.3 TCP Sliding Window

Offsetadressen im Bytestrom



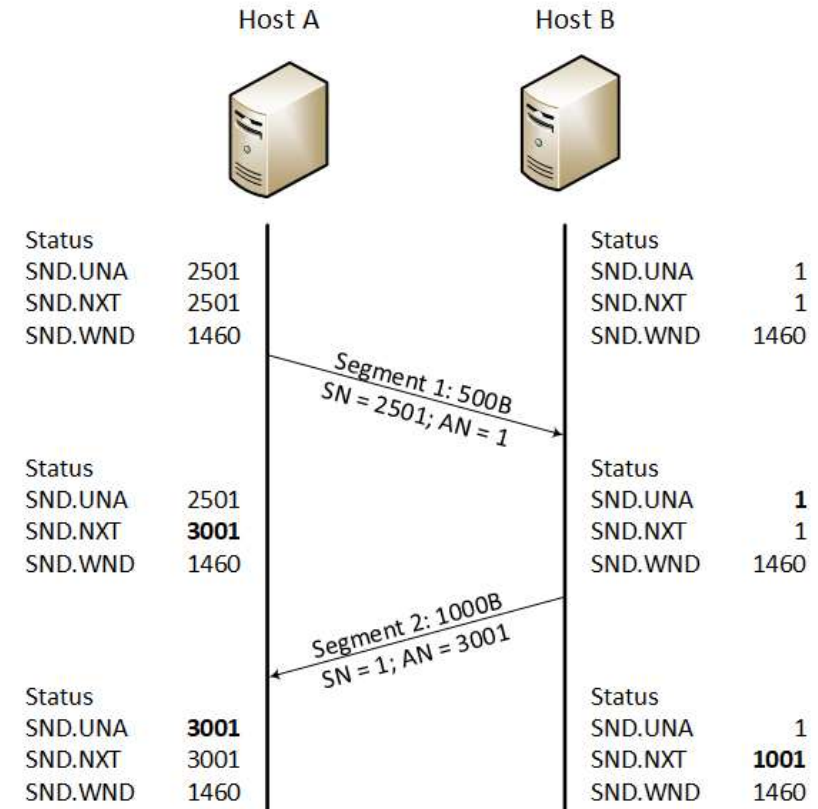
- Abschnitte
 - 1: Diese Daten sind bereits gesendet und bestätigt
 - 2: Diese Daten sind bereits gesendet aber noch nicht bestätigt
 - 3: Diese Daten dürfen noch gesendet werden
 - 4: Diese Daten dürfen noch nicht gesendet werden
- Das Sendefenster besteht also aus Abschnitten 2 und 3

6.3 TCP Sliding Window

Ablauf der Datenübertragung

- (Relative) Sequenznummer bestimmt den Offset im Nutzdatenstrom
- Wenn der Sender jetzt ein Segment wegschickt, hat dieses Segment die Sequenznummer SND.NXT
- Wenn der Empfänger alle offenen Daten bestätigen möchte, verwendet er als AN ebenfalls SND.NXT

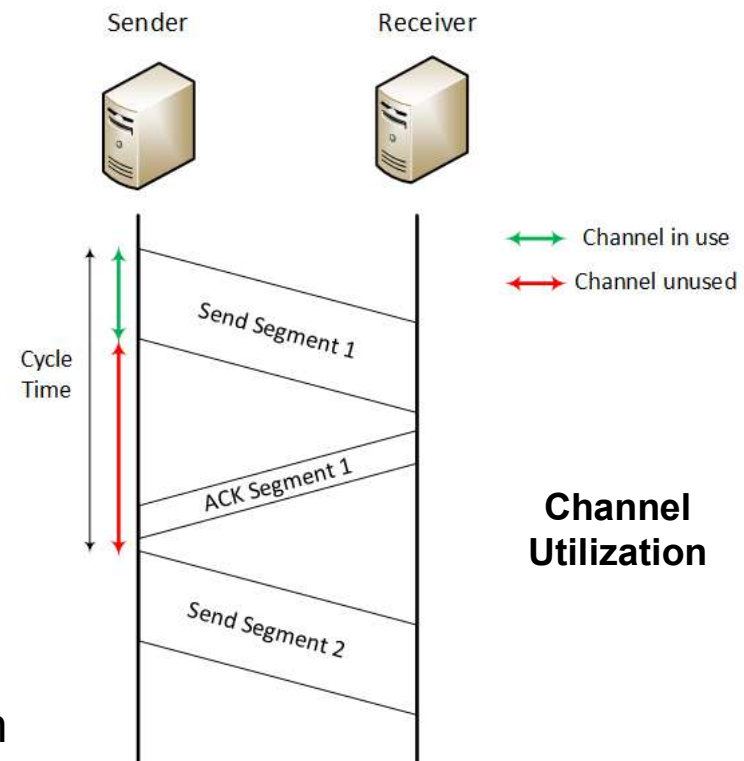
Zustandsvariablen
im Zeitverlauf



6.3 TCP Sliding Window

Festlegung der Fenstergröße

- GoBackN und Selective Repeat verwenden fixe Fenstergrößen → Frage: Was wäre die optimale Fenstergröße?
- Im fehlerlosen Fall sollte die Channel Utilization (genutzte Zeit durch Gesamtzeit) gleich 1 sein (keine Leerstände)
- Problem: Bei End-to-end Verbindungen kann die RTT sehr lang sein → Sehr große Fenster sind aber problematisch bei Fehlern



6.3 TCP Sliding Window

Fenstergrößen bei TCP

- TCP verwendet daher variable Fenstergrößen; diese Fenster nehmen auf Fehlersituationen, aber auch Lastsituationen (Überlaststeuerung) Rücksicht
- Die Fenstergröße wird beschränkt durch
 - Flow Control: Explizite Information durch den Empfänger mithilfe des TCP Headerfelds WND (der Empfänger kann das beliebig herunterregeln)
 - Congestion Control: Implizite Information durch fehlende Bestätigung für gesendete Segmente (hier geht auch die Fehlersituation ein)
- Das Minimum der beiden Werte wird als Sendefenster (Kreditrahmen) verwendet

6.3 TCP Sliding Window

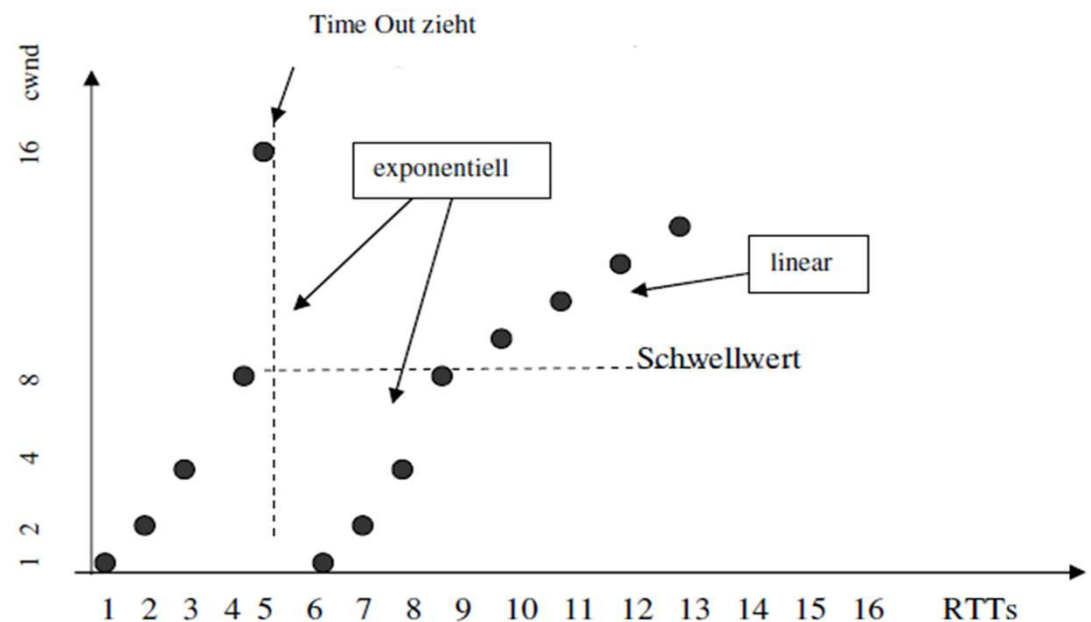
TCP Congestion Control

- Congestion Control ist der primäre Begrenzungs-Mechanismus bei TCP (sorgt dafür, dass Fenster nicht zu groß werden)
 - Congestion Window ist immer gesetzt (kann nicht ∞ werden)
 - Congestion Window ist adaptiv (wird durch Fehler verkleinert – unabhängig von der Fehlerursache)
- Der Sender beginnt mit einer geringen Fenstergröße und vergrößert dann die Fenstergröße sukzessive, so lange keine Fehler auftreten
 - „Slow Start TCP“ startet mit 1 MSS (typischerweise 1460 Byte)
 - Vergrößerung wird durchgeführt bei jedem bestätigten Segment

6.3 TCP Sliding Window

Verlauf des Congestion Windows

- Nach einem Fehler fängt TCP wieder bei einer MSS an
- Durch die Beschränkung des WND Felds auf 16 bit ist ein theoretisches Maximum gegeben (65535 MSS)
- Das Wachstum ist zunächst exponentiell, dann ab einem Schwellwert linear



Zeitlicher Verlauf des Congestion Windows